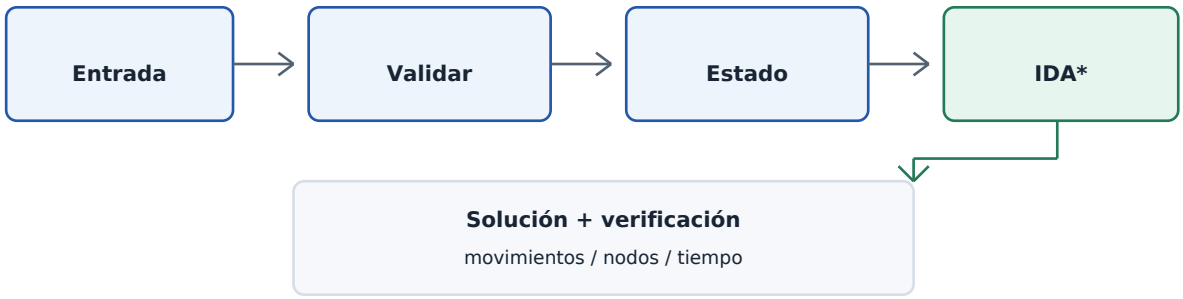
POCKET CUBE 2×2×2

Tutorial para comprender el prototipo, ejecutar el solucionador y defenderlo oralmente

La idea central

El programa no memoriza una solución para cada cubo. Representa el estado actual con sus ocho esquinas y utiliza búsqueda informada IDA* para encontrar una secuencia de movimientos que lo devuelva al estado resuelto.



Qué contiene	Para qué sirve
Modelo de esquinas	Representar las piezas, sus posiciones y su orientación.
Tablas de patrones	Dar una estimación segura de los movimientos que faltan.
IDA*	Explorar soluciones cada vez más largas sin guardar todo el árbol.
Verificación y pruebas	Comprobar que la secuencia obtenida realmente resuelve el estado.

Objetivo de estudio: no memorices todo el código línea a línea. Aprende a explicar el estado, la heurística, el bucle de búsqueda y cómo se valida el resultado.

Proyecto individual · Python 3 · Sin dependencias externas

1. Cómo representa el cubo

Un 2×2×2 tiene ocho piezas de esquina. Cada una lleva tres pegatinas, por lo que el modelo trabaja con ocho posiciones y tres valores por posición. No necesita modelar centros ni aristas porque el Pocket Cube no los tiene.



Cada posición guarda tres pegatinas en un orden fijo.

Dos datos describen el estado

Dato	Qué significa	Ejemplo conceptual
perm (permutación)	Qué pieza ocupa cada posición.	En URF está la pieza que originalmente era DFR.
ori (orientación)	Cómo está girada esa pieza en su esquina.	La pieza de URF tiene giro 1 (una de las dos orientaciones alternativas).

El código usa dos tuplas de ocho números: `perm` y `ori`. Si el cubo está resuelto, `perm = (0,1,2,3,4,5,6,7)` y todas las orientaciones valen cero.

¿Por qué la orientación suma 0 módulo 3?

Al girar caras, las esquinas pueden cambiar de orientación, pero el giro total siempre conserva la regla `sum(ori) % 3 == 0`. Por eso solo hacen falta los giros de siete esquinas: el de la octava queda determinado por los otros siete.

> Idea que conviene decir en voz alta: “El estado se separa en permutación y orientación. Así el algoritmo no tiene que comparar 24 pegatinas en cada decisión; trabaja con una representación compacta de las ocho esquinas.”

2. Cómo se aplican los movimientos

Se admite la notación habitual: U, R, F, D, L y B. La letra indica la cara que se gira. Un apóstrofo significa giro antihorario y el sufijo 2 significa media vuelta. Cada uno de los 18 movimientos posibles cuenta como un paso de búsqueda.

Notación	Significado	Ejemplo
R	Giro horario de la cara derecha, visto desde fuera.	R
R'	Giro antihorario de la cara derecha.	R'
U2	Media vuelta de la cara superior.	U2
F / D / L / B	La misma regla aplicada a otra cara.	F D L B

Qué hace el modelo al aplicar un movimiento

Cada movimiento contiene dos tablas: `source_for_dest`, que indica de qué posición viene cada esquina, y `delta_for_dest`, que indica el giro adicional introducido en el destino. La nueva permutación y la nueva orientación se calculan así:

```
new_perm[dest] = perm[source]
new_ori[dest]  = (ori[source] + delta[dest]) % 3
```

¿De dónde salen las tablas?

El programa crea una geometría de 3 ejes para cada esquina y sus pegatinas. Gira las coordenadas de las cuatro esquinas de una cara y traduce el resultado a permutación y orientación. Los giros inversos y de 180° se construyen componiendo ese movimiento. Así no hay que escribir a mano cada cambio de pieza.

> Prueba importante: los tests comparan dos implementaciones independientes: girar pegatinas y actualizar (perm, ori). Si no coinciden para secuencias aleatorias, hay un error en las tablas de movimiento.

3. Por qué se usa IDA*

Una búsqueda explora posibles secuencias de movimientos. El reto es evitar probar millones de secuencias inútiles y, a la vez, conservar la posibilidad de encontrar una solución corta.

Método	Ventaja	Problema para este prototipo
BFS (anchura)	Encuentra la solución mínima si todos los movimientos cuestan 1.	Guarda demasiados estados al aumentar la profundidad.
DFS (profundidad)	Usa poca memoria.	Puede profundizar por un camino malo y no garantiza la solución mínima.
A*	Ordena la frontera por g+h y suele explorar menos.	Necesita guardar una gran frontera de estados.
IDA*	Prueba límites crecientes de g+h y usa memoria moderada.	Puede volver a explorar estados en distintos límites.

Los dos valores clave

Símbolo	En el programa
g	Profundidad actual: movimientos que ya se han aplicado en esta ruta.
h	Estimación mínima de movimientos que aún faltan.
f = g + h	Coste estimado de la ruta completa. La búsqueda no expande rutas que superan el límite actual.

Pseudocódigo simplificado

```
limite = h(estado_inicial)
MIENTRAS limite <= profundidad_maxima:
    buscar recursivamente con f = g + h
    si estado resuelto: devolver camino
    si no: aumentar limite al menor f que se ha pasado
fallar con mensaje si se alcanza el máximo
```

Por qué puede encontrar la solución más corta: los límites aumentan en orden y la heurística nunca sobreestima lo que falta. Con los podados seguros del código, la primera solución encontrada pertenece al menor límite alcanzable.

4. La heurística y las tablas de patrones

Si IDA* solo supiera que cada movimiento cuesta uno, tendría que probar demasiadas alternativas. La heurística le da información sobre lo lejos que está el estado de la solución. Para no descartar la solución óptima, la estimación debe ser una cota inferior: nunca puede decir que faltan más movimientos de los que realmente faltan.

El programa construye dos tablas por búsqueda en anchura (BFS):

Tabla	Estados de la proyección	Qué mide
Permutación	8! = 40.320	Distancia mínima para ordenar las ocho piezas, ignorando su orientación.
Orientación	3^7 = 2.187	Distancia mínima para orientar las esquinas, ignorando qué pieza ocupa cada lugar.

La heurística combinada

```
h(estado) = max(dist_perm[perm], dist_ori[ori])
```

Se utiliza el máximo, no la suma. Un mismo giro puede mover esquinas y cambiar orientaciones simultáneamente, por lo que sumar ambas distancias podría sobreestimar. El máximo sigue siendo una cota inferior segura.

¿Qué no son estas tablas?

No contienen una solución completa para cada estado real del cubo. Son tablas de distancias para dos versiones simplificadas del problema. Aportan una estimación rápida; IDA* sigue teniendo que buscar la secuencia de movimientos en el estado completo.

> Una frase para el profesor: “La heurística combina dos proyecciones: ordenar piezas y orientar esquinas. Como cualquiera de las dos distancias es un mínimo necesario, su máximo no sobreestima y permite mantener la optimalidad de IDA*.”

En tiempo de arranque se generan las tablas pequeñas. En las ejecuciones siguientes del menú se reutilizan en memoria; no se vuelve a construir la base en cada intento de resolución.

5. Cómo leer el código sin perderse

Zona del archivo main.py	Responsabilidad	Qué explicar
CORNER_FACES y vectores	Define las ocho esquinas, caras y coordenadas.	Qué significa URF, DFR, etc.
_build_sticker_permutation	Genera el giro geométrico de una cara.	Cómo se mueve una pegatina de origen a destino.
decode_stickers / encode_stickers	Convierte entre 24 pegatinas y (perm, ori).	Validación de piezas repetidas y suma de orientaciones.
_build_move_definitions	Construye los 18 movimientos.	Cómo se extraen los cambios de permutación y giro.
_build_pattern_databases	Calcula distancias de las proyecciones por BFS.	Por qué 40.320 y 2.187 estados.
PocketCubeSolver.solve / _search	Realiza IDA*.	Límite, heurística, poda y condición de objetivo.
main()	Menú y experiencia de uso.	Cómo introducir un scramble o las pegatinas.

Validaciones para evitar respuestas falsas

Antes de resolver, el programa comprueba que cada una de las ocho combinaciones de colores corresponde a una esquina conocida, que no se repite ninguna pieza, que cada terna tiene un orden cíclico posible y que la suma de orientaciones es cero módulo 3. Si falla algo, muestra un error de entrada en vez de inventar una solución.

Podas seguras para reducir duplicados

No se repite inmediatamente la misma cara (por ejemplo, R seguido de R puede combinarse en un único R2 o simplificarse). Además, los giros de caras opuestas conmutan: U y D afectan a capas distintas. Se fija un orden canónico para no explorar tanto U-D como D-U. La tabla de estados visitados incluye también la cara anterior, ya que esa cara afecta a qué continuaciones se permiten.

> Importante para aprender: puedes modificar mensajes y menú con poco riesgo; evita cambiar las convenciones de orientación o la lógica de IDA* sin volver a ejecutar las pruebas.

6. Cómo ejecutarlo y probarlo con un cubo físico

En Windows, descomprime el ZIP y ejecuta `run_windows.bat` o abre una terminal en la carpeta y escribe `python main.py`. Necesitas Python 3.10 o superior. No hay que instalar paquetes.

Flujo de prueba recomendado

Paso	Acción
1	Selecciona la opción 1 y genera un scramble corto de 5 a 8 movimientos.
2	Aplícalo al cubo físico partiendo de un cubo resuelto. La secuencia impresa es el SCRAMBLE.
3	Copia los movimientos de la SOLUCIÓN en el mismo orden y aplícalos al cubo.
4	Comprueba que queda resuelto y que el programa indica “Verificación interna: CORRECTA”.
5	Repite con 9 o 10 movimientos y explica por qué la solución puede ser más corta que el scramble.

Resolver un estado que ya tienes

Opción 3: introduce las pegatinas de cada esquina según su posición. Por ejemplo, para URF se introducen, en este orden, el color de la pegatina de U, la de R y la de F. Por defecto: W=blanco, Y=amarillo, G=verde, B=azul, R=rojo, O=naranja. Usa el mismo mapa de colores que tenga tu cubo; si es distinto, actualiza DEFAULT_COLOR_MAP.

Opción 2: si conoces los movimientos con los que se ha mezclado el cubo desde un estado resuelto, introdúcelos separados por espacios. El programa reconstruye el estado y lo resuelve.

> Consejo práctico: mañana, para demostrar que entiendes la búsqueda, empieza por una mezcla corta que puedas verificar a mano. Luego pasa a una longitud mayor si el tiempo lo permite.

Ejemplo de formato de entrada: `R U R' F2 D L'`. La notación usa apóstrofo recto para los giros inversos.

7. Preguntas típicas y respuestas claras

¿Qué algoritmo de búsqueda has utilizado?
IDA*, una búsqueda en profundidad con umbrales crecientes para g+h.

¿Qué es g?
La profundidad actual: cuántos movimientos lleva la ruta explorada.

¿Qué es h?
Una estimación inferior del número de movimientos que faltan para resolver el estado.

¿Por qué IDA* y no BFS?
BFS puede almacenar demasiados estados al crecer la profundidad. IDA* controla la memoria con búsquedas repetidas limitadas por f.

¿Por qué usas dos tablas?
Una mide distancia de permutación; la otra, distancia de orientación. Son proyecciones pequeñas que aportan información útil.

¿Por qué combinas las tablas con max y no con suma?
Porque un mismo movimiento puede mejorar ambas cosas a la vez. La suma podría sobreestimar y romper la garantía de la heurística.

¿Cómo sabes si la solución funciona?
Aplico la secuencia al estado inicial representado y compruebo que permutación y orientación coinciden exactamente con el estado resuelto.

¿La solución es siempre la más corta?
Con la heurística admisible, umbrales crecientes y podas que eliminan solo rutas equivalentes, la primera solución está a la menor profundidad encontrada. El programa tiene un máximo configurable de profundidad.

¿Qué complejidad tiene?
En el peor caso, IDA* puede crecer exponencialmente con la profundidad. La heurística y la poda reducen el trabajo, pero no eliminan el peor caso.

¿Qué limitación reconoces?
La búsqueda puede tardar más en estados difíciles y la entrada manual exige registrar correctamente las 24 pegatinas en el orden indicado.

> Si te atascas: explica primero el estado (perm y ori), después la heurística (máximo de dos distancias) y por último el bucle de límites. Es el hilo conductor del proyecto.

8. Repaso final antes de la prueba

Glosario rápido

Término	Significado sencillo
Estado	Descripción completa de las posiciones y orientaciones actuales.
Permutación	Qué esquina se encuentra en cada posición.
Orientación	Cómo está girada cada esquina en su posición.
Heurística admisible	Estimación que nunca es mayor que el coste mínimo real que falta.
Tabla de patrones (PDB)	Distancias exactas dentro de una representación simplificada del problema.
IDA*	Búsqueda en profundidad que repite la exploración aumentando el límite de g+h.
Nodo	Un estado que la búsqueda considera durante su exploración.
Poda	Regla que evita explorar alternativas redundantes sin perder soluciones válidas.

Lista de comprobación

☐ Puedo explicar qué contienen perm y ori.

☐ Puedo interpretar R, R' y U2.

☐ Puedo explicar g, h y f=g+h sin mirar el PDF.

☐ Recuerdo por qué se usa el máximo de las distancias.

☐ Sé ejecutar el programa y distinguir SCRAMBLE de SOLUCIÓN.

☐ Puedo explicar cómo se verifica una solución y para qué sirven los tests.

> Resumen en una frase: “Modelo las ocho esquinas, genero los cambios de cada movimiento, calculo dos tablas de distancia y aplico IDA* con una heurística admisible hasta encontrar una secuencia que deja el estado exactamente resuelto.”

Archivos del proyecto: `main.py` (prototipo), `tests.py` (pruebas), `README.md` (instrucciones rápidas). Ejecuta `python -m unittest -v` para repetir la batería de pruebas.